ShardFit: Vision and Engineering Brief

Status: v1, 3 November 2026. Audience: the ShardFit engineering team. Purpose: give the team everything needed to build the project without further input from the founder. Working name: ShardFit (check for name collisions before any public use).

1. What ShardFit is

ShardFit is a runtime that lets one small GPU run open-source language models that are larger than its memory, **losslessly**, by splitting the model into shards and streaming them through VRAM. On top of it sits an **unattended agent runner**: the user hands over a task, the system works through it in a loop with no human prompts, and returns a result bundle at the end.

It trades speed for capacity. It is slower than a cloud API and slower than a model that fits in memory, but it runs privately on hardware the user already owns.

One line: *Run bigger open-source models than your GPU can hold, unattended.*

2. Why it exists

- Consumer GPUs have 6 to 24 GB of VRAM. Useful open models range from 10 GB to hundreds of GB.
- Today a user can (a) pick a smaller or more heavily quantized model, (b) use partial offload (for example llama.cpp layer offload), or (c) rent cloud compute.
- Partial offload works but its speed is hard to predict, and nobody tells the user up front what will run and how fast.
- Our gap: **predictable streaming with a planner that says what will run and how fast, plus a resumable unattended runner for overnight and batch agent work.**

3. What it is NOT (non-goals)

- Not a chat or autocomplete engine. Streaming speeds are too low for that on big models.
- **No layer skipping, no shard skipping, no routers, no bridge networks.** Every shard of the loaded model runs on every pass. The only approximation allowed is quantization, and it must always be disclosed. The earlier Hop6 research design is archived and out

of scope.

- Not a claim to beat existing offload tools on raw speed. That must be measured first (Phase 0).
- Not a claim of unlimited capability. See sections 8, 9 and 14.

4. Principles

1. **Lossless relative to the loaded model.** Output must match a reference implementation running the same weights.
2. **Measure before claiming.** No speed, quality or completion-rate number is published without hardware, model, quantization and method attached.
3. **Predict before running.** The planner estimates tokens per second before the user commits hours of compute.
4. **Report failures.** Benchmark tables include failed runs and negative results.
5. **Unattended must be safe.** Budgets, sandboxing, checkpoints and a human review gate are mandatory, not optional.

5. How it works

5.1 Shards

A shard is a contiguous block of one or a few transformer layers. If a single layer is bigger than a streaming slot, split it into attention and MLP sub-blocks. Each shard is stored as one contiguous buffer so a transfer is one large copy, not hundreds of small ones.

5.2 VRAM budget

Let V be total VRAM. Reserve R for the KV cache (at the target context length and batch), activations, CUDA context and workspace. Weights get W = V - R. Allocate S streaming slots (at least 2, for double buffering) of size C each. The remaining Pin = W - S*C holds shards that stay resident permanently. Always keep the embedding and output head resident.

5.3 Execution loop

While the compute stream runs shard i, a separate copy stream loads shard i+1 into a free slot from pinned host memory (non-blocking copies, CUDA events for synchronization). Slots rotate. Resident shards are simply executed in place.

5.4 Storage tiers

Shards live in one of three tiers: GPU (pinned), host RAM (pinned), NVMe (memory-mapped or direct IO with read-ahead). The scheduler assigns tiers from measured bandwidths. Streaming from NVMe is several times slower than from RAM, so RAM is filled first.

5.5 Planner

Given a model file, a hardware profile and a context length, the planner predicts tokens per second and gives a go or no-go before the run starts (formulas in section 6). The planner is a core feature, not a tool: it is how we avoid over-promising.

5.6 Mixture-of-experts path

For MoE models, only the shared weights plus the experts a token routes to need to be read. Pin shared weights, keep a cache of experts in VRAM and RAM, and count misses. Which experts are needed is known only after the router runs, so exact prefetching is not possible: use a cache plus an optional predictor, and group small transfers.

5.7 More tokens per pass

Streaming cost is per pass over the weights, so anything that yields more tokens per pass helps: prefill (processes the whole prompt in one pass), batching several sequences, and speculative decoding with a small resident draft model verified by the streamed model.

5.8 Agent runner

A separate component that drives the engine through a task loop. Specified in section 13.

6. The math

Symbols (sizes in GB, bandwidths in GB/s):

- M: model size as stored (quantized size if quantized)

- V: total VRAM, R: reserved, W = V - R
- S: number of slots, C: shard size, Pin = W - S*C
- B = M - Pin: bytes streamed per pass
- L: effective bandwidth from where the weights live to the GPU
- v: VRAM bandwidth
- T: tokens produced per pass (1 for plain decoding)

```
t_pass = max( B / L ,  M / v )
tok/s  = T / t_pass          (usually ~ T*L / B, since L << v)

Largest model for a speed target tok_min:
M_max = (W - S*C) + L*T / tok_min
```

Conditions that must also hold:

1. C <= W / S, and a shard is at least one layer or sub-block.
2. If B exceeds the host RAM you can pin (H), the rest streams from SSD: t_load = H/L + (B - H)/SSD.
3. M must fit in RAM plus SSD capacity.

MoE variant. With shared weights A_sh, total expert bytes E, k of N experts active, and an expert cache hit rate h:

```
M_act = A_sh + (k/N)*E
B_tok = (1 - h) * (k/N) * E
tok/s = T / max( B_tok / L , M_act / v )
E_max = L*T / ( tok_min * (1 - h) * (k/N) )
```

Real decode speed is typically 60 to 80 percent of these ceilings. The planner must be calibrated against measurements.

7. Expected speeds (estimates, not measurements)

All numbers below come from the formulas above with stated assumptions and must be validated in Phase 2.

RTX 3050 6GB laptop (assumed: about 168 GB/s VRAM, PCIe 4.0 x8 with about 12 GB/s effective, R about 1.5 GB, W about 4.5 GB, S=2, C=0.5 GB, T=1). Verify the exact variant.

Model	Estimated tok/s
3-4B, 4-bit (about 2.5 GB), fits	45 to 60
8B, 4-bit (about 4.7 GB), barely fits	20 to 28
14B, 4-bit (about 8.5 GB), about half streamed	2 to 3
7-8B fp16 (14 to 16 GB), streamed	about 1
333B dense, 4-bit (about 185 GB) from NVMe	about 0.03 (roughly 35 s per token)

Largest streamable dense model on this machine, weights in RAM: about 9.5 GB at 2 tok/s, 15.5 GB at 1 tok/s, 27.5 GB at 0.5 tok/s. At one token every 12 seconds (5 tokens per minute): about 63 GB from NVMe, or about 147 GB if that much RAM is available to pin.

Mac M5 16GB (unified memory, 153 GB/s listed by Apple, about 12 GB usable by the GPU). There is no PCIe copy, so streaming from RAM adds nothing. Models that fit run at bandwidth speed: 8B 4-bit about 20 to 28 tok/s, 14B 4-bit about 11 to 14 tok/s. Models over about 12 GB page from the SSD and drop to a fraction of a token per second.

MoE example (assumed k/N = 1/16, h = 0.3, T = 1, target 5 tok/s): E_max is about 55 GB with experts in pinned RAM (roughly a 100B-parameter MoE at 4-bit) and about 23 GB with experts on NVMe. A dense model at the same speed would be about 6 GB.

Estimates for agent latency: a 1,000-token answer at 5 tok/min takes about 3.3 hours. Reasoning models that emit thousands of thinking tokens are far worse.

8. What it is good for and not good for

Good for

- Overnight and batch agent work where total throughput matters more than per-token latency.
- Read-heavy, write-light jobs: code review verdicts, classification, repo summarization, choosing which file to edit. Prefill is efficient, so a large input costs about the same as a small one.
- Batches of independent jobs (test generation, docstrings, labeling, migration reviews), because one pass over the weights serves many sequences.
- Running a large model as an occasional planner or judge next to a small fast resident model.

- Private, local use of models that otherwise would not run.

Not good for

- Interactive chat on large streamed models.
- Inline autocomplete.
- Tight human-in-the-loop coding where each iteration needs a fast answer.
- Long outputs on large dense models (time scales with output length).

9. Positioning and claims

Approved positioning: run open-source models larger than your GPU memory, unattended. Give it a task and it works through it in a loop and returns a result. Slower than a cloud API, private, on hardware you already own. Results depend on the model and GPU used, and we publish them.

Allowed claims (only with a published measurement behind each):

- Model X runs on GPU Y at Z tokens per second.
- The planner predicts speed within a stated error.
- On task set T, model X on GPU Y completed N percent of tasks in H hours.

Not allowed:

- No limit, unlimited, any size, any model.
- Does the complete work. Completion depends on the model and is reported per benchmark.
- Novel architecture. Streaming, offloading and expert caching have prior art (section 15).
- Speed or quality numbers without hardware, model and quantization attached.

10. Build plan

Each phase has an exit criterion. Do not start the next phase until it is met.

Phase 0: baseline. Measure llama.cpp partial offload (including CPU-hybrid mode) on the target machines for three or four reference models. Record tok/s, time to first token and VRAM. *Exit:* a baseline table. *Decision:* build on top of llama.cpp or build a separate engine, based on what the data shows.

Phase 1: machine profiler. A tool that reports real host-to-GPU bandwidth for shard-sized pinned copies, SSD sequential read bandwidth, free VRAM and RAM available for pinning. *Exit:* repeatable results across runs.

Phase 2: planner. Implement the formulas from section 6, dense and MoE. *Exit:* predictions within 25 percent of measurements on at least five model and machine combinations. If not, find out why before building more.

Phase 3: dense streamer. Fixed shards, VRAM reservation, resident set, rotating slots with a copy stream, pinned memory, contiguous buffers. *Exit:* output matches the reference path exactly for the same quantized weights, and measured tok/s is within 25 percent of the planner.

Phase 4: MoE path. Shared weights pinned, expert cache, miss counters, grouped transfers, optional predictor. *Exit:* hit rate versus cache size measured on real prompts; speed matches the MoE planner within 25 percent.

Phase 5: tokens per pass. Batching, then speculative decoding with a resident draft model. Report each gain separately from the streaming gain. *Exit:* measured gains with acceptance rates.

Phase 6: agent runner. Section 13. *Exit:* the agent benchmark in section 11 run end to end on at least two model and machine pairs.

11. Benchmark protocol (required for every milestone)

- Fixed model, fixed machine, fixed prompt set, fixed perplexity dataset. Record exact model names, quantization and software versions.
- Reference models: choose current open-weight models of these classes and record the exact names: a 7 to 8B dense 4-bit control that fits on the 6 GB GPU; a 14B dense 4-bit that needs streaming on 6 GB; a 30B-class MoE; and a 70B-class dense 4-bit for machines with enough RAM.
- Speed metrics: tokens per second, time to first token, prefill time, at three context lengths.
- Quality: perplexity on a fixed dataset, compared with the reference implementation. For the lossless path the difference must be zero within numerical tolerance.
- Agent metrics: completion rate, total wall-clock time and tokens, on a fixed set of 20 to 50 coding or data tasks with automatic pass or fail (for example unit tests). Run a small resident-model control on the same tasks.
- Report failures and crashes, not only successes.

12. Decision gates

- If Phase 0 shows llama.cpp already matches ShardFit's planned speed, build the planner and agent runner on top of it instead of a new engine.
- If planner predictions are off by more than 25 percent, fix the model before continuing.
- If the best model that meets the speed target completes only a small share of benchmark tasks, the agent claim is not ready to market. Publish the data and adjust the claim, not the benchmark.
- If the lossless output ever differs from the reference, stop and fix before any other work.

13. Agent runner specification

- **Task input:** a goal, allowed tools, working directory, and budgets.
- **Loop:** plan, act, observe, repeat, until the task is judged complete or a budget is hit. No human prompts during the run.
- **Resumable:** checkpoint after every step. A restart or power loss must resume from the last checkpoint, because runs last hours.
- **Budgets:** hard limits on steps, tokens and wall-clock time, with a clean stop and a partial result when hit.
- **Sandboxed:** runs in a container, no network by default, file access limited to the working directory.
- **Logged:** full transcript of every step, tool call and output.
- **Result bundle:** final output or diff, artifacts, the step log, and a run report (model, quantization, GPU, total time, tokens, tool failures).
- **Human review gate:** the runner proposes changes. It does not push, deploy, send or delete anything outside the sandbox without explicit confirmation.
- **Failure handling:** detect loops and repeated failed steps, and stop rather than burn hours.

14. Risks and limitations

1. Latency compounds in agent loops: 30 model calls take 30 times the per-call time.
2. The KV cache lives in VRAM and grows with context, shrinking the weight budget. Batching multiplies it.

3. Very long prompts are compute-bound on small GPUs during prefill.
4. Quantization lowers quality, noticeably at 2 to 3 bits for code.
5. MoE cache misses cause unpredictable stalls.
6. Sustained NVMe reads heat laptops and can throttle.
7. Agent reliability depends on the model, so a larger model at 0.1 tok/s may still fail tasks a smaller model finishes.
8. Unattended agents can do damage. The sandbox and review gate are safety requirements.
9. Unified-memory machines (Apple silicon) gain nothing from streaming within RAM.
10. Hardware specs in this document are assumptions until measured.

15. Prior art and novelty stance

Layer or expert offloading and streaming already exist (llama.cpp partial offload, FlexGen, AirLLM, DeepSpeed ZeRO-Inference, MoE expert offloading). Dynamic layer skipping and selection also have published work (for example FiRST, D-LLM, DASH, and a paper on dynamic layer selection in decoder-only transformers), but ShardFit does not use skipping. Read these papers and tools before writing anything public.

Our stance: ShardFit is an engineering product, not a research claim. Its defensible value is (1) a calibrated planner that predicts what will run and how fast, and (2) a resumable, sandboxed unattended runner with an honest results table. Do not describe it as a new architecture. If a novelty claim is wanted later, it needs a literature search and a benchmark against the tools above.

16. Decision rights and working agreement

The team decides, without asking: implementation language and framework, whether to build on llama.cpp or separately (subject to the Phase 0 data), shard sizes, scheduler algorithms, repository structure, tooling, which reference models to use within the classes in section 11, and the order of work inside a phase.

Escalate to the founder before doing: any change to the lossless principle (including adding skipping, routing or bridges), any public claim or marketing copy, the project name, the license, dropping the agent runner or the planner, any spend beyond the agreed budget, and anything that would run an agent outside the sandbox.

Proposed cadence (founder to confirm): a short written status every week covering what was measured, what failed, and what is next. The benchmark results table is the source of truth. If an opinion and the table disagree, the table wins.

17. Glossary

- **Shard:** a contiguous block of model layers (or sub-blocks) moved as one unit.
- **Resident / pinned shard:** a shard kept in VRAM permanently.
- **Slot:** a VRAM buffer that streamed shards rotate through.
- **Prefill:** processing the input prompt in one pass.
- **Decode:** generating output tokens one pass at a time.
- **KV cache:** stored attention keys and values for the context; lives in VRAM and grows with context.
- **MoE:** mixture of experts, where each token uses only some of the model's experts.
- **Speculative decoding:** a small model proposes several tokens and the big model verifies them in one pass.
- **Lossless:** output identical to the same weights run without streaming.

18. Open questions to settle by measurement

1. What is the real effective host-to-GPU bandwidth on the target 6 GB laptop (PCIe lane width and generation)?
2. Does CPU-hybrid execution beat GPU streaming on these machines, given system RAM bandwidth?
3. What expert-cache hit rate do real prompts achieve versus cache size on the chosen MoE?
4. What is the speculative decoding acceptance rate on coding tasks?
5. Which reference models complete a useful share of the agent benchmark at an acceptable total time?